

MLOps gobernado por especificaciones (SDD)

Marco de adopción para el ciclo de vida de modelos en Databricks · Prueba técnica Machine Learning Engineering

1. Problema y tesis

Un equipo de Data Science ejecuta de forma manual el entrenamiento, la validación, el registro y la promoción de modelos a producción. El riesgo no es la ausencia de criterio técnico, sino que ese criterio vive en personas y notebooks: no es reproducible, no es auditable y no escala a varios modelos ni a varios equipos.

La organización ya cuenta con lineamientos que describen qué debe cumplirse en cada fase del ciclo. **Un lineamiento es una norma; una especificación es una norma verificable por máquina.** La propuesta consiste en convertir cada contrato descrito en prosa en un artefacto versionado que el pipeline pueda leer, validar y hacer cumplir, de modo que ningún modelo alcance producción sin evidencia de haber satisfecho sus condiciones.

Principios de diseño

- **La especificación es la fuente de verdad.** Un cambio de comportamiento se hace cambiando la spec, nunca ajustando manualmente un recurso en la plataforma.
- **Fallo temprano y explícito.** Un contrato incumplido detiene el pipeline; nunca produce un artefacto silenciosamente degradado.
- **Parametrización total.** Ambiente, cómputo, catálogo e identidad son parámetros del *target*, no decisiones incrustadas en la lógica.
- **Promoción, no reentrenamiento.** Producción recibe el artefacto ya validado, no uno equivalente.
- **Evidencia por diseño.** Métricas, veredictos, linaje y trazas se generan como subproducto de la ejecución.

2. Arquitectura del marco

Capas de especificación

Contrato	Artefacto	Declarar	Punto de control
Producto	openspec/config.yaml	Dominio, área, modelo, catálogos, identidad	Convenciones de nombres y scaffolding
Datos	data_contract	+	Bloquea la extracción si no cumple
Features	expectations		
	definitions.py	Catálogo de variables, tipos, origen	Misma lógica en entrenamiento e inferencia
Entrenamiento	train.py	Features, target, partición temporal, semillas	Reproducibilidad y linaje en MLflow
Validación	thresholds.py	Umbral, segmentos críticos, tolerancia vs. campeón	Verdicto PASS/FAIL; asigna @challenger
Despliegue	databricks.yml	Targets, cómputo, run_as, permisos	Gate de aprobación previo a @champion
Monitoreo	quality_monitors	Drift, desempeño, segmentos, cadencia	Alerta y dispara reentrenamiento

Table 1: Contratos del ciclo de vida y su mecanismo de cumplimiento.

Flujo de promoción entre ambientes

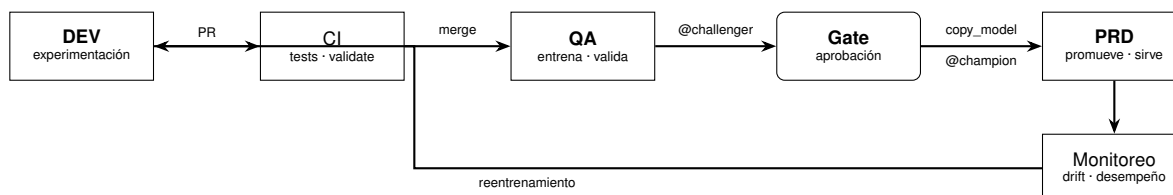


Figure 1: Flujo de promoción. El entrenamiento está excluido de PRD por diseño.

Ambientes e identidad

Cada ambiente corresponde a un catálogo de Unity Catalog (`gold_adv_{dev,qa,prd}`), que es la frontera natural de permisos y auditoría. El mismo *bundle* se despliega en los tres; solo cambian las variables del *target*: catálogo, cómputo, identidad y permisos.

La ejecución en QA y PRD se realiza con **service principal**, nunca con identidad de usuario: un job atado a una persona se rompe cuando esa persona cambia de equipo, y no ofrece segregación de funciones. El principal se declara en la spec (`run_as`) junto con los roles mínimos requeridos, de modo que la solicitud al equipo de identidad sea un artefacto revisable en el *pull request* y no un ticket informal. Para la conexión entre Azure DevOps y la plataforma se recomienda federación de identidad de carga de trabajo (OIDC), que elimina secretos de larga duración.

3. Mecanismo de ejecución

Empaquetado declarativo

El proyecto se empaqueta como **Databricks Asset Bundle**: un archivo `databricks.yml` declara jobs, experimentos, modelos registrados y monitores, y el CLI los materializa. Ningún recurso productivo se crea manualmente en la interfaz. Esto produce tres propiedades: el recurso en producción es derivable del repositorio, cualquier ambiente es reconstruible desde cero, y toda modificación queda sujeta a revisión.

El cómputo es un parámetro del *target*. Donde existe capacidad provisionada se referencia el clúster corporativo; donde se prioriza elasticidad se omite el campo y la ejecución ocurre en cómputo *serverless*. La lógica y los contratos no cambian.

Pipelines de CI/CD

Pipeline	Disparador	Responsabilidad
ci	Pull request	Estilo, pruebas unitarias sobre lógica pura, <code>bundle validate</code> , análisis estático de seguridad. Bloquea el <i>merge</i> si falla.
cd-training	Merge a principal	Despliega el bundle en QA y ejecuta el ciclo: features, entrenamiento, validación contra umbrales y registro con alias <code>@champion</code> en el catálogo de QA.
cd-scoring	Rama de versión	Despliega en PRD. Tras aprobación manual en un <i>environment</i> protegido, ejecuta la copia del modelo validado y habilita inferencia y monitoreo.

Table 2: Pipelines y separación de responsabilidades entre ambientes.

Patrón de promoción

Se adopta el patrón de **implementación de código**: el código de entrenamiento se promueve entre ambientes y el modelo se entrena con los datos correspondientes a cada uno. Sobre él se aplica una restricción adicional: **el entrenamiento está excluido del target de producción**. Producción no entrena; recibe mediante `copy_model_version` la versión que ya obtuvo veredicto PASS en QA, preservando `run_id` y etiquetas de trazabilidad.

La consecuencia es que el artefacto en producción es idéntico —no equivalente— al que superó la validación, y que no existe ruta para registrar un modelo en producción sin atravesar el gate de aprobación.

Monitoreo y reentrenamiento

Cada ejecución de inferencia escribe en una **tabla de inferencia** gobernada por Unity Catalog, con la predicción, la marca temporal y el identificador de la versión del modelo que la produjo. Esa columna permite reconstruir qué versión generó qué decisión, lo que convierte la tabla en evidencia de auditoría y no solo en un registro operativo.

Sobre esa tabla se declara un monitor que calcula perfil estadístico de variables, deriva entre ventanas temporales y —cuando el resultado real está disponible— desempeño observado. Los segmentos monitoreados son los mismos declarados como críticos en el contrato de validación, de modo que el criterio de aceptación y el criterio de vigilancia hablan del mismo negocio y se mantienen en un solo lugar. La superación de un umbral de deriva o la caída de desempeño por debajo del mínimo constituyen disparadores de reentrenamiento.

Consumo asistido por agentes

Las predicciones se exponen al negocio mediante un agente conversacional que consulta el modelo vigente y su tabla de inferencia a través de herramientas declaradas. El agente opera bajo el mismo principio de gobierno: solo lee, cada cifra que reporta procede de una consulta verificable, y toda interacción queda registrada con sus llamadas a herramientas, latencia y consumo. El gobierno no se detiene en el modelo; la capa que lo consume también requiere contrato y traza.

4. Roles y responsabilidades

Rol	Responsable de	Límite
Científico de datos	Contratos de datos, features, entrenamiento y umbrales; experimentación en DEV	No promueve ni opera producción; acceso de solo lectura a PRD
Ingeniero de ML	Bundle, pipelines, operación de QA y PRD, monitoreo y reentrenamiento	No define umbrales de negocio ni aprueba por sí mismo
Plataforma Seguridad	Creación de service principals, asignación de roles, políticas de clúster y red	No modifica lógica ni contratos del producto
Dueño de negocio	Criterios de aceptación y aprobación de promoción a producción	No interviene en la implementación

Table 3: Separación de funciones. Ningún rol concentra definición, validación y promoción.

5. Ruta de evolución

Fase	Objetivo	Resultado verificable
1	Versionar y parametrizar	El entrenamiento manual se reescribe como código sin valores incrustados; se registra en MLflow con linaje.
2	Declarar contratos	Esquema, features, umbrales y segmentos críticos quedan en archivos versionados; el pipeline falla si no se cumplen.
3	Empaquetar y automatizar	El proyecto se despliega como bundle en DEV y QA; la CI bloquea el merge ante contrato incumplido.
4	Gobernar la promoción	PRD deja de entrenar; el modelo llega por copia desde QA, tras aprobación, ejecutado con service principal.
5	Cerrar el ciclo	Monitoreo sobre tablas de inferencia, disparadores de reentrenamiento y consumo asistido con trazabilidad.

Table 4: Adopción incremental. Cada fase entrega valor sin requerir la siguiente.

6. Riesgos, dependencias y controles

Riesgo	Efecto	Control
Configuración temporal que alcanza producción (umbrales relajados, datos de prueba)	Modelo sin validación real operando sobre decisiones de negocio	Precondiciones verificadas en el despliegue de PRD: el pipeline rechaza el target si un parámetro está fuera del rango declarado
Fuga de parametrización en librerías compartidas	Rompe la portabilidad entre ambientes sin generar error visible	Pruebas que despliegan el bundle en un ambiente alterno; una ruta incrustada falla de inmediato
Fuga temporal en el conjunto de entrenamiento	Métricas optimistas, degradación inmediata en producción	Declaración explícita de qué campos existen al momento de la inferencia; partición temporal obligatoria
Etiqueta real con rezago	El desempeño observado no puede calcularse en la ventana corriente	Monitoreo de deriva como señal anticipada; desempeño recalculado al llegar la etiqueta
Dependencia del equipo de identidad	Bloquea el despliegue en QA y PRD	Identidades y roles declarados en la spec como requisito explícito, solicitados por PR con antelación

Table 5: Riesgos priorizados y mecanismos de control asociados.

7. Supuestos y límites

- Se asume Unity Catalog habilitado, separación por catálogo entre ambientes y Azure DevOps como plataforma de CI/CD, con cómputo ya provisionado.
- La creación de service principals y la asignación de roles corresponden al equipo de plataforma, no al equipo de ML. El marco lo asume y declara la identidad requerida como parte de la especificación.
- El marco gobierna el ciclo de vida; no sustituye el criterio de modelado. Define dónde se declaran los umbrales, no cuáles son: esos provienen del negocio.
- El patrón descrito asume inferencia por lotes como caso predominante. El servicio en tiempo real está contemplado y se habilita por parámetro del target, pero exige resolución de variables en línea y un régimen de latencia y costo distinto.
- La automatización no elimina la aprobación humana en la promoción a producción. La reduce a una decisión informada por evidencia, en lugar de un acto de confianza.